코드가 읽히기 시작하는
바이브 코딩 복습 노트

DAY01 - DAY29

확인된 23일의 수업을 한 권으로

핵심 개념 · 실행 흐름 · 직접 수정 연습
복습문제 · 해설 · 실제 자료 출처

비개발자 창업가를 위한 목표
AI가 만든 코드에서 바꿀 위치를 찾고, 결과를 예측하고, 직접 확인하기

분석·편집: 2026-11-12
DAY09 및 DAY22-26 수업 본문 미확보
원본 교재 전체를 합친 자료가 아니라, 확인한 자료에 기반한 요약·복습본

이 책의 구성

01 확인 범위와 학습 기준

02 DAY01 첫 수업: HTML 읽기

03 일자별 핵심 요약과 수정 연습 - 23일

04 코드 흐름 상세 연습 - 입력, 목록, API, React

05 수정 위치 찾기와 복습문제 해설

06 추가 자료 활용, 복습 순서, 출처 목록

읽는 방법

각 DAY에서 먼저 실행 흐름을 읽고, 수정 연습의 결과를 예상해 보세요. 실제로 고친 뒤 예상과 결과를 비교하고, 마지막에 해설을 확인하면 됩니다.

DAY 번호는 수업 순서입니다. 실제 수업 날짜와 학습 완료 여부를 뜻하지 않습니다.

1. 실제로 확인한 범위

Downloads의 「1인창업가 캠프 수업자료」 폴더를 찾았다. 하위 폴더 없이 PDF 28개가 들어 있으며, 총 1,282쪽이다. 구성은 DAY별 교재 23개, 생성형 AI 기초 3개, CSR·SSR 부록 1개, DAY10~13 핵심복습 1개다.

28개 모두 텍스트를 추출해 문서 구조를 확인했다. DAY별 학습 목표와 정리 부분을 대조했고, React DAY28~29의 주요 코드와 설명, Notion 과제 본문을 중심으로 분석했다. 모든 1,282쪽의 이미지와 실습을 하나씩 시각 검수하거나 실행한 것은 아니다. PDF 추출문에는 일부 띄어쓰기·특수문자 오류가 있어, 아래 예제는 읽기 쉽게 다시 작성했다.

Notion의 데일리 과제와 하위 과제 23개 본문을 조회했다. DAY29에 첨부된 cafe-order_day29_시작.zip도 별도 작업 공간에 받아 압축 내부 파일을 읽었다. 수업자료를 덮어쓰지 않았고, 수업 프로젝트의 의존 패키지를 설치하거나 앱을 실행하지 않았다. ZIP의 Header.jsx가 실제로 count를 받는 것까지 확인했다.

수업 자료의 존재와 본인의 완료 진도는 다르다. 조회한 과제는 TODO와 제출 안내 중심이다. 학생이 완성한 코드·제출물·이해도는 확인하지 않았다. 따라서 아래 내용은 확인된 수업 순서에 맞춘 복습 지도이며, DAY29까지 모두 숙달했다는 평가가 아니다.

확인되지 않은 부분:

• DAY9, DAY22~26: 이 로컬 폴더에는 해당 DAY 교재가 없고, 조회한 데일리 과제 목록에도 없다. 어떤 수업을 했는지 채워 넣지 않았다.

• DAY22 학습 기록: 1인 창업가 캠프 day22는 2026-11-03 날짜와 달성 표시가 있지만 본문은 비어 있다. 수업 내용이나 제출 완료의 근거로 삼지 않았다.

• DAY20 학습 기록: 1인창업가 캠프 day 20은 2026-10-30 날짜와 달성 표시가 있으나 본문이 비어 있다. 데일리 과제의 DAY20 본문은 별도로 읽었다.

• OT 원본: 북마크에서 원본 페이지를 따라갔으나 NOT_FOUND가 반환됐다. 삭제 여부와 접근 권한 문제 중 어느 것인지는 단정할 수 없다.

• 실제 수업 날짜: DAY 번호와 달력 날짜 전체를 연결하는 출석표·시간표는 확보하지 못했다. 날짜를 임의로 배정하지 않았다.

• DAY30 이후: DAY29 PDF의 다음 수업 예고는 확인했지만, 해당 수업 본문을 확보한 것은 아니다.

• 첨부 자료: DAY29 시작 ZIP만 내부를 읽었다. 나머지 과제 첨부파일 전체를 내려받아 확인한 것은 아니다.

2. 이 수업을 배우는 목적에 맞춘 기준

목표는 AI 코드에서 다음 다섯 가지를 찾는 것이다.

1. 무엇을 보여 주나? HTML·JSX에서 제목, 카드, 입력칸을 찾는다.

2. 무슨 데이터를 쓰나? 변수·객체·배열·state를 찾는다.

3. 무엇이 시작시키나? 첫 실행, 클릭, 폼 제출, Effect 중 무엇인지 찾는다.

4. 어떻게 바뀌나? 조건·계산·함수·서버 요청을 따라간다.

5. 어디에 결과가 반영되나? 화면, 브라우저 저장소, 서버를 구분한다.

수업은 다음처럼 연결된다.

HTML로 내용과 입력칸을 만든다
→ CSS로 모양과 배치를 정한다
→ JavaScript로 데이터를 계산한다
→ 클릭·입력에 함수를 연결한다
→ 서버에서 받고 브라우저에 저장한다
→ React에서 state와 props로 화면을 연결한다

DAY01 첫 수업: HTML 읽기

오늘 목표는 화면에서 바꾸고 싶은 부분을 코드에서 찾는 것입니다. 웹 브라우저가 서버에 요청하면 서버가 HTML을 응답하고, 브라우저가 문서를 읽어 화면에 표시합니다. HTML은 제목·문단·이미지·링크처럼 내용과 구조를 적는 언어입니다.

```
<!DOCTYPE html>
<html lang="ko">
  <head>
    <meta charset="UTF-8">
    <title>동네 카페</title>
  </head>
  <body>
    <h1>오즈 카페</h1>
    <p>커피와 빵을 판매합니다.</p>
    <a href="menu.html">메뉴 보기</a>
  </body>
</html>
```

head는 문서 정보와 설정, body는 본문입니다. title은 브라우저 탭 제목이고 h1은 본문의 가장 중요한 제목입니다. p는 문단, a는 링크입니다. 메뉴 보기에서 href는 이동 주소, 태그 사이의 '메뉴 보기'는 보이는 글자입니다. 글자만 '주문하기'로 바꾸어도 이동 주소는 여전히 menu.html입니다.

직접 답해 보기

1. 탭에 '나의 첫 홈페이지'를 표시하려면 어디를 고칠까요?

2. 본문의 '오즈 카페'를 '종현 카페'로 바꾸려면 어디를 고칠까요?

3. 링크를 누르면 order.html로 이동하게 하려면 무엇을 바꿀까요?

확인

1번은 title 안의 글자, 2번은 h1 안의 글자, 3번은 href의 값을 바꿉니다. 보이는 문구와 동작을 정하는 속성을 구분하는 것이 첫 번째 읽기 연습입니다.

3. 일자별 요약·실행 흐름·직접 수정 연습

아래 '수정 연습'과 '복습문제'는 원본 과제의 복사가 아니라, 확인된 개념에 맞춰 이번 분석에서 새로 구성했다. 문제 해설은 뒤에 모았다. 원본 과제명은 해당 Notion 페이지로 연결된다.

DAY01 웹의 동작 구조와 HTML 기초

핵심 개념 head는 문서 정보, body는 본문. 태그에 속성으로 주소·설명을 붙인다.

실행 흐름 서버에 요청 → HTML 응답 → 브라우저가 문서를 해석한다. 로컬 HTML은 서버 없이도 열 수 있다.

직접 수정 <title>과 <h1>을 서로 다른 문구로 고치고 탭·본문의 차이를 본다.

복습문제 탭 제목만 바꾸려면 어느 태그를 고칠까?

메모 / 예상 결과: _____

DAY02 회원가입 폼 만들기

핵심 개념 action·method·name, label, 입력 type, required

실행 흐름 입력 → 브라우저 검증 → 정상적인 폼 제출이라면 지정 주소로 값 전송. 화면만 있어도 가입 처리가 구현된 것은 아니다.

직접 수정 이메일 입력칸에 required를 넣고 빈 값·잘못된 형식을 시험한다.

복습문제 id는 있는데 name이 없으면 일반 폼 전송에서 어떤 문제가 생길까?

메모 / 예상 결과: _____

DAY03 시맨틱 구조로 다시 쓰기

핵심 개념 header·nav·main·article·footer, 제목 계층, alt

실행 흐름 문서 구조를 브라우저와 보조기기가 읽는다. 의미 있는 태그가 구조를 설명한다.

직접 수정 클릭하는 동작을 div 대신 실제 button으로 표현하고 키보드로 이동해 본다.

복습문제 작은 글씨가 필요하다는 이유로 h1을 h4로 바꿔도 될까?

메모 / 예상 결과: _____

DAY04 선택자로 스타일 입히기

핵심 개념 외부 CSS 연결, .class·#id, 후손·자식 선택자, 우선순위

실행 흐름 HTML의 대상 → 일치하는 CSS 규칙 → 겹치는 규칙을 판단 → 최종 모양

직접 수정 전체 button 대신 .order-button의 배경색만 고친다.

복습문제 색을 고쳤는데 그대로라면 무엇부터 확인할까?

메모 / 예상 결과: _____

DAY05 소개 영역과 NEW 배지 만들기

핵심 개념 content·padding·border·margin, Flexbox·Grid, position

실행 흐름 요소 크기 계산 → 부모 배치 규칙 적용 → 자식 배치. 배지의 위치는 기준 조상에 좌우된다.

직접 수정 카드 안쪽 공간은 padding, 카드 사이 거리는 부모의 gap으로 바꿔 본다.

복습문제 width 200, padding 좌우 20씩, border 좌우 5씩이면 기본 박스의 너비는?

메모 / 예상 결과: _____

DAY06 소개 영역 반응형과 다크 모드

핵심 개념 viewport, 상대 단위, 미디어 쿼리, 모바일 퍼스트

실행 흐름 좁은 화면 기본 규칙 → 조건에 맞는 미디어 쿼리 추가 적용

직접 수정 모바일 1열, 768px 이상 2열로 만들고 경계 폭 전후를 비교한다.

복습문제 min-width: 768px는 390px 화면에 적용될까?

메모 / 예상 결과: _____

DAY07 내 페이지를 GitHub에 공개하기

핵심 개념 변경 확인, stage·commit·push, 브랜치·PR·충돌

실행 흐름 파일 수정 → 변경 내용 확인 → 기록할 변경 선택 → 로컬 기록 → 원격 업로드

직접 수정 문구 한 줄만 고친 뒤 변경 비교 화면에서 그 한 줄만 바뀌었는지 본다.

복습문제 commit만 하면 GitHub의 코드도 바뀔까?

메모 / 예상 결과: _____

DAY08 Bootstrap으로 블로그 글 페이지 만들기

핵심 개념 준비된 CSS 클래스, container·row·col, 반응형 접두사, 모달

실행 흐름 라이브러리 연결 → 클래스에 맞는 스타일 → 모달은 연결된 JS와 대상 id로 동작

직접 수정 col-lg-8·col-lg-4를 col-lg-6·col-lg-6으로 바꾼다.

복습문제 색은 적용되는데 모달이 안 열리면 어디를 확인할까?

메모 / 예상 결과: _____

DAY10 콘솔에 책 정보 출력하기

핵심 개념 const·let, 자료형, typeof, 오류 종류

실행 흐름 값 선언 → 계산·문자열 조합 → Console 출력

직접 수정 가격이 문자열인지 확인하고 Number()를 거쳐 계산한다.

복습문제 "4000" + 500의 결과는?

메모 / 예상 결과: _____

DAY11 놀이공원 입장료 계산기

핵심 개념 =와 ===, &&·||, if·else if, 삼항식

실행 흐름 입력값 → 조건을 위에서부터 판정 → 맞는 분기 계산 → 결과

직접 수정 할인 기준을 바꾸고 기준 바로 아래·같은 값·바로 위를 시험한다.

복습문제 age >= 10 다음 else if (age >= 20)이면 25세는 어느 쪽으로 갈까?

메모 / 예상 결과: _____

DAY12 용돈 저금 계산기

핵심 개념 반복문, 매개변수·인수, 함수 호출, return

실행 흐름 함수 호출 → 인수 받기 → 반복·계산 → return → 호출한 자리에서 결과 사용

직접 수정 월 저축액과 목표액을 바꾸고 반복이 종료되는 이유를 설명한다.

복습문제 함수 안에서 출력만 하고 return하지 않으면 호출 결과는?

메모 / 예상 결과: _____

DAY13 카페 메뉴판 다루기

핵심 개념 배열은 순서, 객체는 속성, 객체 배열, JSON 변환

실행 흐름 JSON 문자열 → JSON.parse → 배열·객체 읽기 → 항목 변경·출력

직접 수정 메뉴 객체의 가격만 바꾸고 다른 메뉴가 유지되는지 본다.

복습문제 두 번째 메뉴의 이름은 menus[2].name일까?

메모 / 예상 결과: _____

DAY14 영화 관람 기록 정리하기

핵심 개념 map 변환, filter 선별, find 첫 항목, reduce 집계

실행 흐름 배열 → 조건으로 고름 → 필요한 모양으로 바꿈 → 합계·목록

직접 수정 평점 기준을 바꾸고 먼저 남을 영화 목록을 예측한다.

복습문제 항목 4개를 map으로 바꾸면 보통 결과 개수는? filter는?

메모 / 예상 결과: _____

DAY15 여행 준비물 목록 만들기

핵심 개념 DOM, 요소 선택, textContent, classList, 요소 생성·추가

실행 흐름 데이터 → 요소 만들기 → 내용 채우기 → 화면에 붙이기

직접 수정 챙긴 준비물에 붙는 CSS 클래스의 색과 취소선을 고친다.

복습문제 querySelector가 null이면 다음 줄에서 무엇을 점검할까?

메모 / 예상 결과: _____

DAY16 물 마시기 기록 버튼 만들기

핵심 개념 이벤트와 콜백, input·change·submit, preventDefault

실행 흐름 클릭 → 연결 함수 호출 → 잔 수 변경 → 목표 판정 → 화면 반영

직접 수정 1잔 증가를 2잔 증가로 바꾼 뒤 목표 도달 조건도 확인한다.

복습문제 이벤트에 handleClick과 handleClick()을 넘기는 차이는?

메모 / 예상 결과: _____

DAY17 도서 대출 확인 만들기

핵심 개념 Promise, async·await, 성공·실패, 타이머

실행 흐름 확인 시작 → 기다리는 동안 나머지 작업 → 완료·실패 후 이어서 실행

직접 수정 대기 중 문구를 바꾸고 완료 전·후 화면을 구분한다.

복습문제 await를 만나면 브라우저의 모든 JavaScript가 멈출까?

메모 / 예상 결과: _____

DAY18 글 한 편 보기 만들기

핵심 개념 요청 주소·메서드·응답 상태, fetch, response.ok, JSON, 오류 처리

실행 흐름 요청 → 응답 상태 검사 → JSON 읽기 → 화면 반영 / 실패 안내 → 마무리

직접 수정 글 번호를 바꾸고 정상·없는 글·연결 실패를 구분한다.

복습문제 서버가 404를 보내면 fetch는 반드시 바로 catch로 갈까?

메모 / 예상 결과: _____

DAY19 음료 자판기 만들기

핵심 개념 스코프·클로저, 호출 방식과 this, 고차 함수, Date·reduce

실행 흐름 버튼 → 동전을 더하거나 구매 함수 실행 → 잔액 확인·차감 → 판매 기록 → 화면

직접 수정 물 가격 800원을 1000원으로 바꿀 때 버튼 글자와 makeBuyer에 넣는 가격을 함께 확인한다.

복습문제 makeBuyer("물", 800)이 연결 시 호출되는데 왜 바로 구매되지 않을까?

메모 / 예상 결과: _____

DAY20 읽은 글 표시하기

핵심 개념 서버 데이터와 로컬 기록 구분, JSON 저장, 상태별 화면

실행 흐름 저장된 읽음 번호 복원 + 서버 글 목록 요청 → 목록 그림 → 클릭 시 저장 → 새로고침 후 복원

직접 수정 읽음 스타일을 바꾸고 '모두 지우기'가 해당 키만 지우는지 읽는다.

복습문제 변수의 배열만 바꾸고 저장하지 않으면 새로고침 뒤에는?

메모 / 예상 결과: _____

DAY21 오늘의 메뉴 고르기

핵심 개념 import·export, 기본·이름 내보내기, 모듈, npm·package.json·Vite

실행 흐름 모듈 가져오기 → 버튼 연결 → 메뉴 배열을 함수에 전달 → 선택 결과 표시

직접 수정 menu.js의 배열에 메뉴 하나를 더하고 개수 표시·선택 후보가 같이 바뀌는지 본다.

복습문제 기본 내보내기와 이름 내보내기는 import 중괄호가 어떻게 다를까?

메모 / 예상 결과: _____

DAY27 오즈 카페 — 프로젝트 시작

핵심 개념 Node.js·npm·nvm, scripts, 개발 서버, 프로젝트 파일 구조

실행 흐름 프로젝트 준비 → 의존 패키지 설치 → 개발 서버 → index.html → main.jsx → App

직접 수정 App.jsx의 소개 문구만 바꾸고 화면 반영을 확인한다.

복습문제 npm install과 npm run dev는 각각 무슨 일을 할까?

메모 / 예상 결과: _____

DAY28 오즈 카페 — 메뉴 카드

핵심 개념 JSX, 컴포넌트, props, 구조분해, 기본값

실행 흐름 App에서 메뉴별 값을 넘김 → 같은 MenuCard가 서로 다른 내용으로 표시됨

직접 수정 아메리카노 가격 하나만 바꿀 때 App의 props를 고친다. 모든 카드 모양은 공통 컴포넌트·CSS에서 고친다.

복습문제 desc를 생략한 경우와 desc={null}은 기본값이 같을까?

메모 / 예상 결과: _____

DAY29 오즈 카페 — 담기

핵심 개념 state·set 함수·스냅샷, 불변성, Effect·의존성·정리 함수, 공통 부모

실행 흐름 담기 → 자식이 전달받은 함수 호출 → App의 state 갱신 요청 → 새 값으로 렌더 → Header 갱신 → Effect로 탭 제목 동기화

직접 수정 setCount(c => c + 2)로 바꾸고 초기화 버튼을 추가한다. 제목도 state를 따라가는지 확인한다.

복습문제 같은 이벤트에서 setCount(count + 1)을 두 번 부르면 2가 늘까?

메모 / 예상 결과: _____

4. 코드 흐름을 읽는 연습

A. 입력값이 계산 결과로 이어지는 길 — DAY10~16 연결

아래는 개념을 연결한 새 복습 예제다. 원본 코드 그대로가 아니다.

```
<input id="price" type="number" value="4000">
<button id="calc" type="button">포장비 포함 가격</button>
<p id="result"></p>
<script src="main.js"></script>

const input = document.querySelector("#price");
const button = document.querySelector("#calc");
const result = document.querySelector("#result");

function calculate() {
  const raw = input.value.trim();
  const price = Number(raw);
  if (raw === "" || !Number.isFinite(price) || price < 0) {
    result.textContent = "0 이상의 가격을 입력해 주세요";
    return;
  }
  result.textContent = `${price + 500}원`;
}

button.addEventListener("click", calculate);
```

처음에는 요소를 찾고 버튼에 함수를 연결한다. calculate 본문은 클릭 전에는 실행되지 않는다. 클릭하면 입력 문자열을 읽고, 숫자로 바꾸고, 조건을 검사한 다음 500원을 더해 출력한다.

직접 수정: 포장비를 500원에서 1000원으로 바꾸려면 계산식의 500을 바꾼다. 버튼 글자만 바꾸면 계산은 그대로다. 입력값을 숫자로 변환하지 않으면 덧셈이 문자열 이어붙이기가 될 수 있다.

B. 목록을 고르고 합계를 만드는 길 — DAY13~14 연결

```
const menus = [
  { id: 1, name: "아메리카노", price: 4000 },
  { id: 2, name: "라테", price: 4500 },
  { id: 3, name: "소금빵", price: 3500 },
];

const affordable = menus.filter(menu => menu.price <= 4000);
const names = affordable.map(menu => menu.name);
const total = affordable.reduce((sum, menu) => sum + menu.price, 0);
```

원본 3개 → 4000원 이하 2개 → 이름 ["아메리카노", "소금빵"] → 합계 7500원이다. menu는 각 항목을 잠시 부르는 이름이다. filter 조건을 4500원 이하로 바꾸면 뒤의 이름 목록과 합계가 함께 바뀐다.

이 메서드들이 원본을 직접 바꾸지 않는다는 것과 콜백에서 객체를 직접 바꿔도 된다는 것은 다르다. 예를 들어 map 안에서 menu.price = 0을 하면 원본 객체도 바뀔 수 있다. React에서 가격을 바꿀 때는 새 객체를 만드는 방식으로 연결한다.

```
const updated = menus.map(menu =>
  menu.id === 1 ? { ...menu, price: 4200 } : menu
);
```

C. 서버에서 받은 글과 읽음 기록은 다른 데이터 — DAY18~20 연결

DAY20 원본 과제는 글 목록은 서버에서 받고, 읽은 글 번호만 localStorage에 저장한다.

```
페이지 시작
 ├─ 저장소에서 읽음 번호를 꺼내 배열로 복원
 └─ 서버에 글 목록 요청
     → 응답 상태 검사
     → JSON 본문 읽기
     → posts에 담고 render()

글 클릭
 → 해당 번호가 이미 있는지 확인
 → readIds에 추가
 → JSON 문자열로 저장
 → render()로 읽음 표시
```

이 과제에서 글 제목을 화면에서 바꾸더라도 서버의 글이 수정되지는 않는다. localStorage의 읽음 기록도 다른 기기 계정에 자동으로 동기화되는 데이터는 아니다.

서버 코드에서 반드시 구분해서 읽을 두 번의 기다림:

```
async function getJSON(url) {
  const response = await fetch(url);      // 응답을 기다림
  if (!response.ok) {
    throw new Error(`응답 상태 ${response.status}`);
  }
  const data = await response.json();    // 응답 본문을 읽고 변환
  return data;
}
```

직접 수정: 요청 대상은 URL, 화면 문구는 render, 읽음 보존은 저장 함수에서 찾는다. 모든 수정이 한 파일 안에 있더라도 역할은 다르다. 실패할 때는 Network에서 상태 코드를 보고, Console에서 오류 위치를 함께 본다.

원본 과제의 JSON.parse(localStorage.getItem(KEY)) || []는 키가 없을 때 빈 배열로 시작하게 한다. 저장된 글자가 손상된 경우의 JSON 오류까지 처리하는 코드는 아니다. 이후 보강한다면 복원 부분에 try/catch와 배열 여부 검사를 추가할 수 있다.

D. 최근 과제의 핵심: 담기 버튼부터 탭 제목까지 — DAY29

PDF는 멤버 소개 페이지의 응원 수·발표자 예제이고, Notion 과제는 오즈 카페 주문 앱이다. 두 자료가 다루는 개념은 같지만 파일 이름·변수 이름·화면 결과가 다르다. 아래는 Notion DAY29와 첨부 ZIP에 맞춘 설명이다.

```
App이 count를 기억한다
 ├─ Header에 count를 내려준다 → '담은 메뉴 N개' 표시
 └─ MenuCard에 handleAdd 함수를 onAdd라는 이름으로 내려준다

MenuCard의 담기 버튼 클릭
 → onAdd 호출
 → App의 handleAdd 실행
 → setCount로 다음 값 요청
 → React가 새 count로 컴포넌트를 다시 계산
 → 바뀐 내용을 화면에 반영
 → [count] Effect가 탭 제목을 맞춘다
```

원본 TODO를 채운 주요 줄:

```
const [count, setCount] = useState(0);

useEffect(() => {
  document.title = count > 0 ? `(${count}) 오즈 카페` : "오즈 카페";
}, [count]);

function handleAdd() {
  setCount(count + 1);
}

// App.jsx의 JSX
<Header count={count} />
<MenuCard name="아메리카노" price={4000} onAdd={handleAdd} />

// MenuCard.jsx의 버튼
<button type="button" onClick={onAdd}>담기</button>
```

TODO 1~6은 순서대로 useState, useEffect, count, setCount, count, onAdd다. 다섯 번째 이름은 첨부 ZIP의 Header({ count })와 대조했다. 이 코드는 주요 줄을 모은 발췌이며, 그대로 한 파일로 붙여 실행하는 완성본은 아니다.

용어를 코드와 연결하기

state

이 과제에서 가리키는 것 App이 기억하는 count

고칠 때의 질문 화면을 바꾸며 기억할 값인가?

props

이 과제에서 가리키는 것 Header의 count, MenuCard의 name·price·onAdd

고칠 때의 질문 이 값을 누가 정해서 내려주는가?

Hook

이 과제에서 가리키는 것 useState, useEffect

고칠 때의 질문 컴포넌트 또는 사용자 정의 Hook의 최상위에서 호출하는가?

사이드 이펙트

이 과제에서 가리키는 것 document.title 변경

고칠 때의 질문 JSX 계산 외에 외부와 맞추는 일이 있는가?

의존성 배열

이 과제에서 가리키는 것 [count]

고칠 때의 질문 어떤 값이 달라질 때 다시 맞춰야 하는가?

정리 함수

이 과제에서 가리키는 것 타이머 예제의 return () => clearInterval(timer)

고칠 때의 질문 시작한 타이머·구독 등을 나중에 해제해야 하는가?

여기서 '다시 그린다'는 페이지 전체 새로고침을 뜻하지 않는다. React가 컴포넌트 결과를 다시 계산하고 필요한 DOM 변경을 반영한다는 뜻이다.

직접 수정 1 — 한 번에 두 개 담기

```
function handleAdd() {
  setCount(previous => previous + 2);
}
```

이전 값으로 다음 값을 계산하는 업데이트 함수다. 헤더 글자와 탭 제목은 기존 연결을 통해 따라간다. 따로 숫자를 더하는 코드를 여러 곳에 넣을 필요가 없다.

직접 수정 2 — 초기화 버튼

App의 반환 JSX 안에 추가한다.

```
<button type="button" onClick={() => setCount(0)}>담기 초기화</button>
```

count가 0이면 Header도 0, 탭 제목도 '오즈 카페'가 되어야 한다.

직접 수정 3 — 카드별 수량을 기억하고 싶다면

현재 count 하나로는 총개수만 알 수 있다. 어떤 메뉴를 몇 개 담았는지는 기록하지 않는다. 메뉴별 수량이나 결제 총액까지 구현하려면 메뉴 id·수량을 담는 배열/객체 등으로 데이터 구조를 바꾸어야 한다. 현재 코드만으로 주문·결제 기능까지 구현되었다고 해석하지 않는다.

상태를 바로 읽는 문제

```
setCount(count + 1);
setCount(count + 1);
console.log(count);
```

초기 count가 0인 한 번의 클릭이라면 다음 화면은 1, 같은 핸들러의 출력은 0이다. 이 렌더에서 읽는 count는 스냅샷이기 때문이다. 아래는 2 증가한다.

```
setCount(c => c + 1);
setCount(c => c + 1);
```

Effect에서 읽을 점

• 의존성 배열 생략: 커밋된 렌더 뒤마다 실행한다.

• []: 마운트 때 실행한다. 제거 후 다시 나타나면 또 실행할 수 있다.

• [count]: 처음과 count가 달라진 커밋 뒤 실행한다.

• Effect가 사용하는 반응형 값인 props·state 등을 의존성에서 빠뜨리지 않는다.

• 수업처럼 루트에 StrictMode가 있는 개발 환경에서는 처음에 setup → cleanup → setup을 추가 점검하므로 로그가 두 번 보일 수 있다. 이를 없애려고 무조건 StrictMode를 지우는 방식으로 배우지 않는다.

• 시작한 타이머·구독은 정리 함수에서 해제한다. 탭 제목 대입처럼 모든 Effect에 타이머 정리 코드를 붙여야 하는 것은 아니다.

• JSX에서 계산 가능한 값은 별도 state와 Effect로 복제하지 않는다. const isPopular = cheers >= 3처럼 계산하면 값이 어긋날 일이 줄어든다.

5. 무엇을 바꿀지에 따른 찾는 위치

제목·안내 문구

먼저 찾을 위치 HTML·JSX의 텍스트, 전달되는 props

수정 후 확인 같은 문구가 여러 곳에 중복돼 있는지

카드 하나의 가격

먼저 찾을 위치 메뉴 데이터 또는 App에서 전달하는 price

수정 후 확인 숫자 자료형·표시·계산에 같은 값이 쓰이는지

모든 카드의 테두리·여백

먼저 찾을 위치 공통 CSS 선택자

수정 후 확인 모바일·다크 모드에서도 잘 보이는지

버튼을 누를 때 하는 일

먼저 찾을 위치 이벤트 속성/등록 코드 → 연결 함수

수정 후 확인 첫 실행 때 저절로 실행되지 않는지

목록 필터 기준

먼저 찾을 위치 filter의 조건식

수정 후 확인 경계값과 빈 목록

서버에서 가져올 내용

먼저 찾을 위치 요청 URL·요청 인수·응답 구조

수정 후 확인 Network 상태·반환 데이터

새로고침 뒤에도 유지

먼저 찾을 위치 저장·복원 함수

수정 후 확인 저장 키, JSON 변환, 초기값

여러 컴포넌트가 함께 쓰는 값

먼저 찾을 위치 가장 가까운 공통 부모의 state

수정 후 확인 자식이 콜백으로 알리고 같은 값을 공유하는지

수정은 예상 결과 적기 → 한 군데 변경 → 실행 → 실제 결과 비교 → 변경 내역 확인 순서로 한다. 사용하지 않는 코드처럼 보여도 호출·import되는 곳을 먼저 찾는다. AI가 바꾼 파일이 많다면 요청한 기능과 관련된 변경인지 비교한다.

수업 DAY28~29의 시작 파일 받기에는 강사 버전으로 작업 트리를 교체하는 명령이 있다. 이는 자신의 수정 내용을 그대로 합치는 일반적인 업데이트가 아니다. 직접 적용할 때는 먼저 본인 작업을 기록하고 변경 상태를 확인해야 한다. 이번 분석에서는 이 명령을 실행하지 않았다.

6. 일자별 복습문제 해설

1. DAY01: <title>을 고친다. <h1>은 본문의 제목이다.

2. DAY02: 일반적인 폼 제출의 이름·값 쌍에 포함되지 않는다. id는 label 연결·요소 선택 등에 쓰이고 name이 전송 이름이다.

3. DAY03: 제목 단계는 문서 구조에 맞춘다. 글씨 크기는 CSS로 조정한다.

4. DAY04: CSS 파일이 연결·저장됐는지, 선택자가 실제 요소와 맞는지, 개발자 도구 Styles에서 다른 규칙에 밀렸는지 확인한다.

5. DAY05: 기본 content-box에서는 200+40+10=250px. border-box라면 테두리까지 200px이다. 바깥 margin은 별도다.

6. DAY06: 적용되지 않는다. 768px 이상일 때 조건이 맞는다.

7. DAY07: 아니다. commit은 로컬 기록이고 원격 반영은 push 단계다. 공개 화면 갱신은 별도의 배포 과정이 관련될 수 있다.

8. DAY08: 필요한 Bootstrap JS가 로드됐는지, 모달 버튼의 대상과 모달 id가 일치하는지 확인한다.

9. DAY10: 문자열 "4000500". 숫자로 변환한 뒤 더하면 4500이다.

10. DAY11: 첫 번째 age >= 10 분기다. 더 좁은 조건을 앞에 둘지, 구간을 명시할지 의도에 맞춰 고친다.

11. DAY12: 일반 함수가 끝까지 return 없이 끝나면 undefined. console.log로 출력하는 것과 값을 돌려주는 것은 다르다.

12. DAY13: 두 번째는 menus[1].name. 배열 인덱스는 0부터다.

13. DAY14: 일반적인 4개 항목 배열에 map을 쓰면 4개. filter는 조건에 따라 0~4개다.

14. DAY15: 선택자 오타, 실제 id/class, 요소가 만들어진 시점과 script 위치를 확인한다. null에 속성을 읽거나 쓰면 오류가 날 수 있다.

15. DAY16: handleClick은 나중에 실행할 함수를 전달한다. handleClick()은 지금 실행하고 그 반환값을 전달한다.

16. DAY17: 아니다. 해당 async 함수의 이어지는 부분이 기다린다. 다른 실행 가능한 작업은 계속 진행할 수 있다.

17. DAY18: 아니다. 404·500도 응답을 받은 것이므로 response.ok 등을 검사해 직접 오류로 다뤄야 한다. 연결 실패 등과 구분한다.

18. DAY19: makeBuyer가 구매 동작을 수행할 새 함수를 반환하기 때문이다. 반환된 함수가 나중에 클릭 때 machine.buy를 실행한다. 함수 호출 뒤 무엇을 반환하는지까지 읽어야 한다.

19. DAY20: 마지막으로 저장한 상태로 돌아가거나, 저장 이력이 없으면 초기 상태로 시작한다. 변수 수정만으로는 보존되지 않는다.

20. DAY21: 기본 내보내기는 import pickMenu from "./menu.js", 이름 내보내기는 import { menus } from "./menu.js"처럼 읽는다. 기본 import의 지역 이름은 바꿀 수 있다.

21. DAY27: install은 의존 패키지를 준비하고, run dev는 package.json의 dev 스크립트를 실행한다. 수업 ZIP에서는 Vite 개발 서버를 시작한다.

22. DAY28: 다르다. 매개변수 기본값은 생략하거나 undefined일 때 적용된다. null에는 적용되지 않으며 null 자체는 JSX에서 글자로 표시되지 않는다.

23. DAY29: 같은 클릭에서 같은 스냅샷을 읽으면 1 증가한다. 업데이트 함수를 두 번 쓰면 앞 결과를 이어받아 2 증가한다.

7. 추가 자료를 목적에 맞춰 쓰는 법

새싹반 DAY10~13 핵심복습: Console → 값·자료형 → 조건 → 반복·함수 → 객체 배열·JSON 순서다. React가 막힐 때 전부 다시 시작하기보다, () =>, { ...obj }, items[0].name, return 중 읽히지 않는 부분을 되짚는 자료로 쓴다.

CSR·SSR 부록: 서버에서 HTML을 만들어 보내는지, 브라우저 쪽 JavaScript가 화면을 구성하는지를 비교하는 데 쓴다. '화면이 안 나오는 문제'가 HTML 응답 문제인지, JS 실행·데이터 로딩 문제인지 생각할 출발점이다. SPA는 앱의 화면 전환 구조, CSR·SSR은 렌더링 위치에 관한 구분이므로 완전히 같은 분류로 암기하지 않는다.

생성형 AI 기초 1~3강: AI·머신러닝·딥러닝·생성형 AI의 관계, 기반 모델·LLM·사전/사후 학습, 토큰·다음 토큰 예측·프롬프트를 다룬다. 코드 생성 결과도 실행·수정 검증이 필요하다는 학습 태도와 연결한다. 교재의 제품 예시·가격 관련 일반 설명을 현재 제품 사양으로 검증한 보고서는 아니다.

8. 실제 목적에 맞춘 복습 순서 제안

이 순서는 확인된 수업 진도를 바탕으로 한 제안이다. 실제 숙련도를 측정한 결과는 아니다.

1. 최근 맥락부터: DAY28~29에서 메뉴 하나의 가격 수정 → 담기 +2 → 초기화 → 왜 함께 바뀌는지 설명한다.

2. 문법이 안 읽히면: DAY10~14로 돌아가 객체·배열·콜백·return을 확인한다. 새싹반 요약을 함께 쓴다.

3. 클릭 뒤 흐름이 안 보이면: DAY15~16의 요소 찾기 → 이벤트 → 함수 → 화면 변경을 따라간다.

4. API·저장 문제가 생기면: DAY17~20으로 요청·기다림·오류·저장·복원을 나눠 본다.

5. 디자인 수정이 필요하면: DAY04~06의 선택자·박스·반응형을 집중 확인한다.

6. 변경 전후를 남기기: DAY07의 변경 비교와 commit을 매번 연결한다.

하루 복습은 '설명 5분 + 코드에서 위치 찾기 5분 + 한 군데 수정 10분 + 예상과 결과 비교 5분' 정도로 시작할 수 있다. 넘어가는 기준은 용어를 외웠는지가 아니라 입력 → 함수 → 데이터 변경 → 화면 반영을 자기 말로 설명하고, 작은 변경 결과를 예측할 수 있는지다.

AI에게 수정 요청을 할 때 사용할 문장:

담기 버튼을 한 번 누르면 수량이 2 증가하도록 바꿔 줘. 현재 클릭부터 화면과 탭 제목이 바뀌는 흐름을 먼저 설명하고, 바꿔야 하는 파일과 줄을 알려 줘. 수정 뒤에는 기존 동작과 새 동작을 어떤 방법으로 확인하면 되는지도 설명해 줘.

9. 자료별 출처 목록

아래 로컬 목록은 실제 확인한 PDF의 이름·쪽수·경로다. 일자별 표는 해당 DAY PDF의 목표·정리 부분과 연결된 Notion 과제를 함께 근거로 했다. 코드 흐름 상세 설명은 Notion DAY19~21·DAY27~29, DAY28~29 PDF, DAY29 시작 ZIP을 중심으로 구성했다.

1강 생성형 AI란 무엇인가.pdf · 11쪽

2강 거대 언어 모델(LLM)과 기반 모델의 탄생.pdf · 11쪽

3강 생성형 AI가 실제로 동작하는 방식.pdf · 11쪽

DAY01_HTML문서구조와기본태그.pdf · 46쪽

DAY02_HTML폼과입력요소.pdf · 49쪽

DAY03_시맨틱HTML과웹접근성.pdf · 49쪽

DAY04_CSS개념및선택자.pdf · 44쪽

DAY05_CSS박스모델과레이아웃.pdf · 61쪽

DAY06_반응형웹과미디어쿼리.pdf · 62쪽

DAY07_Git과GitHub.pdf · 61쪽

DAY08_Bootstrap5.pdf · 50쪽

DAY10_JavaScript개요및기본문법.pdf · 62쪽

DAY11_연산자와조건문.pdf · 53쪽

DAY12_반복문과함수.pdf · 47쪽

DAY13_배열과객체기초.pdf · 52쪽

DAY14_배열메서드와데이터처리.pdf · 47쪽

DAY15_DOM조작기초.pdf · 51쪽

DAY16_이벤트처리와폼제어.pdf · 46쪽

DAY17_비동기처리기초.pdf · 51쪽

DAY18_fetchAPI_JSON_오류처리및종합실습.pdf · 51쪽

DAY19_고급함수와고차함수.pdf · 57쪽

DAY20_비동기프로그래밍심화와브라우저저장소.pdf · 58쪽

DAY21_모듈시스템_NPM_Vite.pdf · 52쪽

DAY27_React학습을위한개발환경설정.pdf · 79쪽

DAY28_React개요및JSX.pdf · 41쪽

DAY29_React상태관리기초.pdf · 43쪽

부록_CSR,SSR.pdf · 10쪽

새싹반_DAY10-13_핵심복습.pdf · 27쪽

로컬 원본 위치: Downloads / 1인창업가 캠프 수업자료. Notion 과제 출처는 각 DAY 제목의 링크를 이용하세요.

Notion DAY29 첨부 ZIP의 내부 확인 파일: package.json, src/App.jsx, src/components/Header.jsx, src/components/MenuCard.jsx. 파일 목록도 확인했다. ZIP은 분석 작업 공간에만 보관했고 실행하지 않았다.